JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Interface GatheringByteChannel

All Superinterfaces:

AutoCloseable, Channel, Closeable, WritableByteChannel

All Known Implementing Classes:

DatagramChannel, FileChannel, Pipe.SinkChannel, SocketChannel

---

```
public interface GatheringByteChannel
extends WritableByteChannel
```

A channel that can write bytes from a sequence of buffers.

A *gathering* write operation writes, in a single invocation, a sequence of bytes from one or more of a given sequence of buffers. Gathering writes are often useful when implementing network protocols or file formats that, for example, group data into segments consisting of one or more fixed-length headers followed by a variable-length body. Similar *scattering* read operations are defined in the ScatteringByteChannel interface.

Since:

1.4

Method Summary

All Methods	Instance Methods	

Modifier and Type	Method	Description
long	write (ByteBuffer[] srcs)	Writes a sequence of bytes to this channel from the given buffers.
long	write (ByteBuffer[] srcs, int offset, int length)	Writes a sequence of bytes to this channel from a subsequence of the given buffers.

Methods declared in interface Channel

close, isOpen

Methods declared in interface WritableByteChannel

`write`

Method Details

write

```
long write(ByteBuffer[] srcs,
           int offset,
           int length)
    throws IOException
```

Writes a sequence of bytes to this channel from a subsequence of the given buffers.

An attempt is made to write up to *r* bytes to this channel, where *r* is the total number of bytes remaining in the specified subsequence of the given buffer array, that is,

```
srcs[offset].remaining()
    + srcs[offset+1].remaining()
    + ... + srcs[offset+length-1].remaining()
```

at the moment that this method is invoked.

Suppose that a byte sequence of length *n* is written, where `0` <= *n* <= *r*. Up to the first `srcs[offset].remaining()` bytes of this sequence are written from buffer `srcs[offset]`, up to the next `srcs[offset+1].remaining()` bytes are written from buffer `srcs[offset+1]`, and so forth, until the entire byte sequence is written. As many bytes as possible are written from each buffer, hence the final position of each updated buffer, except the last updated buffer, is guaranteed to be equal to that buffer's limit.

For many types of channels, a write operation will return only after writing all of the *r* requested bytes. Some types of channels, depending upon their state, may write only some of the bytes or possibly none at all. A socket channel in non-blocking mode, for example, cannot write any more bytes than are free in the socket's output buffer. The write method may need to be invoked more than once to ensure that all remaining bytes are written.

This method may be invoked at any time. If another thread has already initiated a write operation upon this channel, however, then an invocation of this method will block until the first operation is complete.

Parameters:

`srcs` - The buffers from which bytes are to be retrieved

`offset` - The offset within the buffer array of the first buffer from which bytes are to be retrieved; must be non-negative and no larger than `srcs.length`

`length` - The maximum number of buffers to be accessed; must be non-negative and no larger than `srcs.length` - `offset`

Returns:

The number of bytes written, possibly zero

Throws:

`IndexOutOfBoundsException` - If the preconditions on the `offset` and `length` parameters do not hold

`NonWritableChannelException` - If this channel was not opened for writing

`ClosedChannelException` - If this channel is closed

`AsynchronousCloseException` - If another thread closes this channel while the write operation is in progress

`ClosedByInterruptException` - If another thread interrupts the current thread while the write operation is in progress, thereby closing the channel and setting the current thread's interrupted status

`IOException` - If some other I/O error occurs

write

```
long write(ByteBuffer[] srcs)
    throws IOException
```

Writes a sequence of bytes to this channel from the given buffers.

An invocation of this method of the form `c.write(srcs)` behaves in exactly the same manner as the invocation

```
    c.write(srcs, 0, srcs.length);
```

Parameters:

`srcs` - The buffers from which bytes are to be retrieved

Returns:

The number of bytes written, possibly zero

Throws:

`NonWritableChannelException` - If this channel was not opened for writing

`ClosedChannelException` - If this channel is closed

`AsynchronousCloseException` - If another thread closes this channel while the write operation is in progress

`ClosedByInterruptException` - If another thread interrupts the current thread while the write operation is in progress, thereby closing the channel and setting the current thread's interrupted status

`IOException` - If some other I/O error occurs

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.